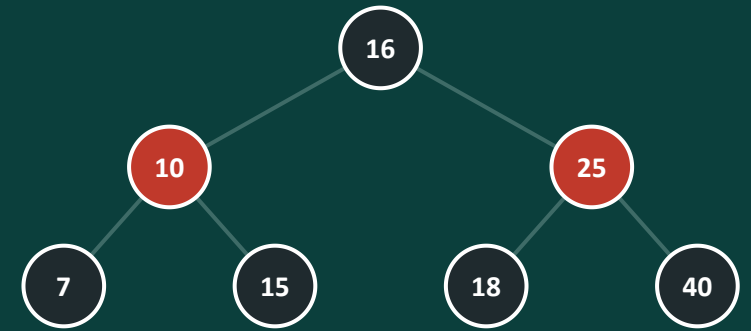




ESTRUTURAS DE DADOS E ALGORITMOS

Árvores de Pesquisa Binária e Árvores Red-Black

Aula 8 · Estruturas de dados avançadas, parte 2 — inserção

Prof. Doutor Luís Simões da Cunha
luiscunha@ismt.pt

Três perguntas para aquecer

1

Qual é a complexidade da pesquisa binária? Que condição exige? (Aula 2)

2

Quanto custa inserir um elemento no meio de um array ordenado? (Aula 1)

3

O que é uma árvore, segundo a definição da Aula 1?

Responda sem olhar para os apontamentos: recordar é o que consolida a memória.

Um milhão de alunos, com matrículas a entrar e a sair todos os dias. Que estrutura usar?

	Pesquisar	Inserir / remover
Array ordenado	$O(\log n)$	$O(n)$
Lista ligada	$O(n)$	$O(1)^*$
Árvore equilibrada	$O(\log n)$	$O(\log n)$

* depois de encontrar a posição, o que já custa $O(n)$

...mas só se a árvore estiver equilibrada. É esse o desafio da aula.

O melhor dos dois mundos

Hoje: a árvore que pesquisa como um array ordenado e insere quase tão bem como uma lista.

Da árvore à árvore que se equilibra sozinha

1

Árvores

Vocabulário e onde se usam.

2

Pesquisa binária

Esquerda menor, direita maior — e o risco de "degenerar".

3

Red-Black

Cinco regras que garantem o equilíbrio.

4

Inserção

Recolorir e rodar, caso a caso.

Pergunta-guia: como garantir que uma árvore nunca fica "torta"?

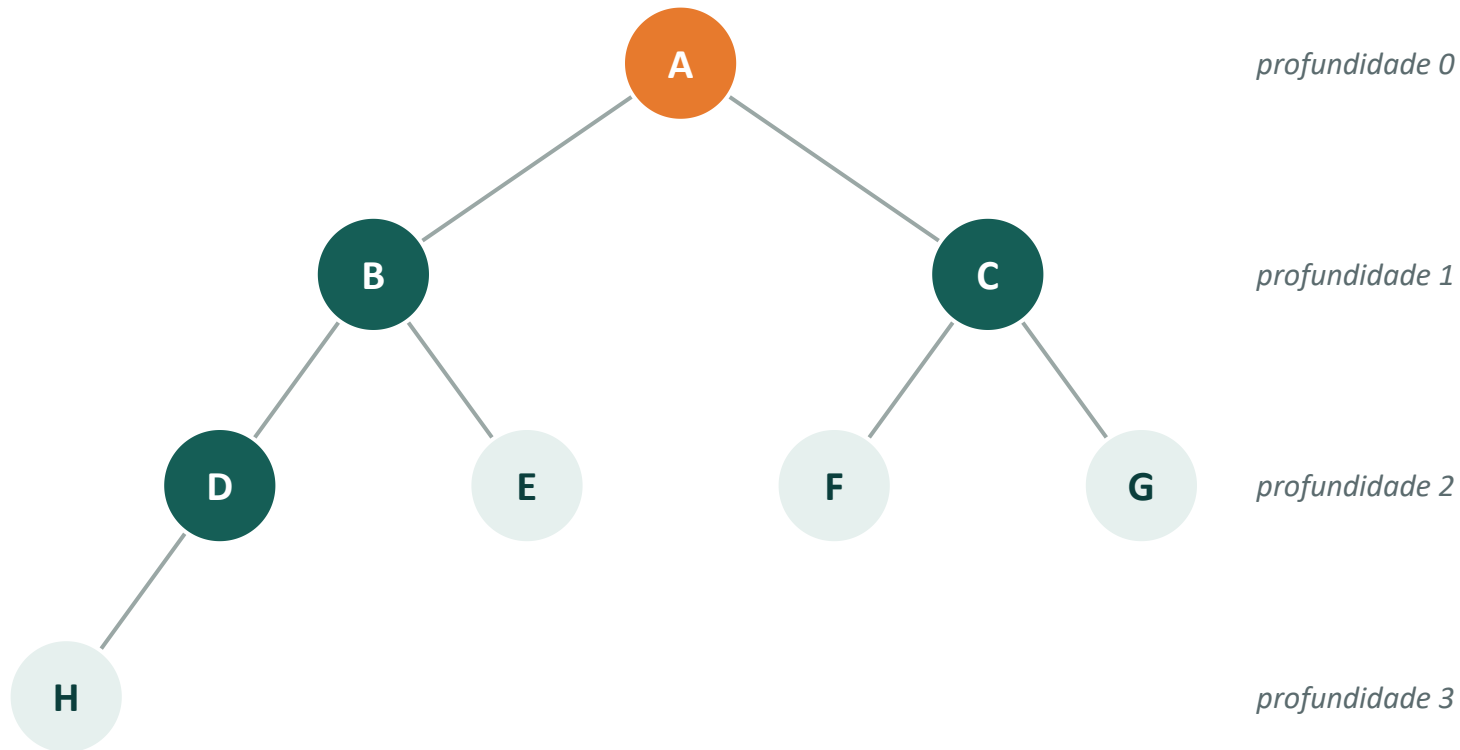
1

PARTE 1

Árvores

Estruturas hierárquicas.

As peças de uma árvore



Raiz
O nó do topo (A). Não tem pai.

Pai e filho
Nós ligados por uma aresta: B é pai de D e E.

Folha
Nó sem filhos (H, E, F, G).

Altura
Número de níveis abaixo da raiz: aqui, 3.

Árvores por todo o lado

1

Sistemas de ficheiros

Pastas dentro de pastas.

2

Bases de dados

Índices para encontrar registos depressa.

3

HTML e XML

A página web é uma árvore de elementos (DOM).

4

Sistemas operativos

Gestão e escalonamento de processos.

5

Compressão

A árvore de Huffman (Aula 6)!

Sempre que há hierarquia ou necessidade de encontrar coisas depressa, há uma árvore escondida.

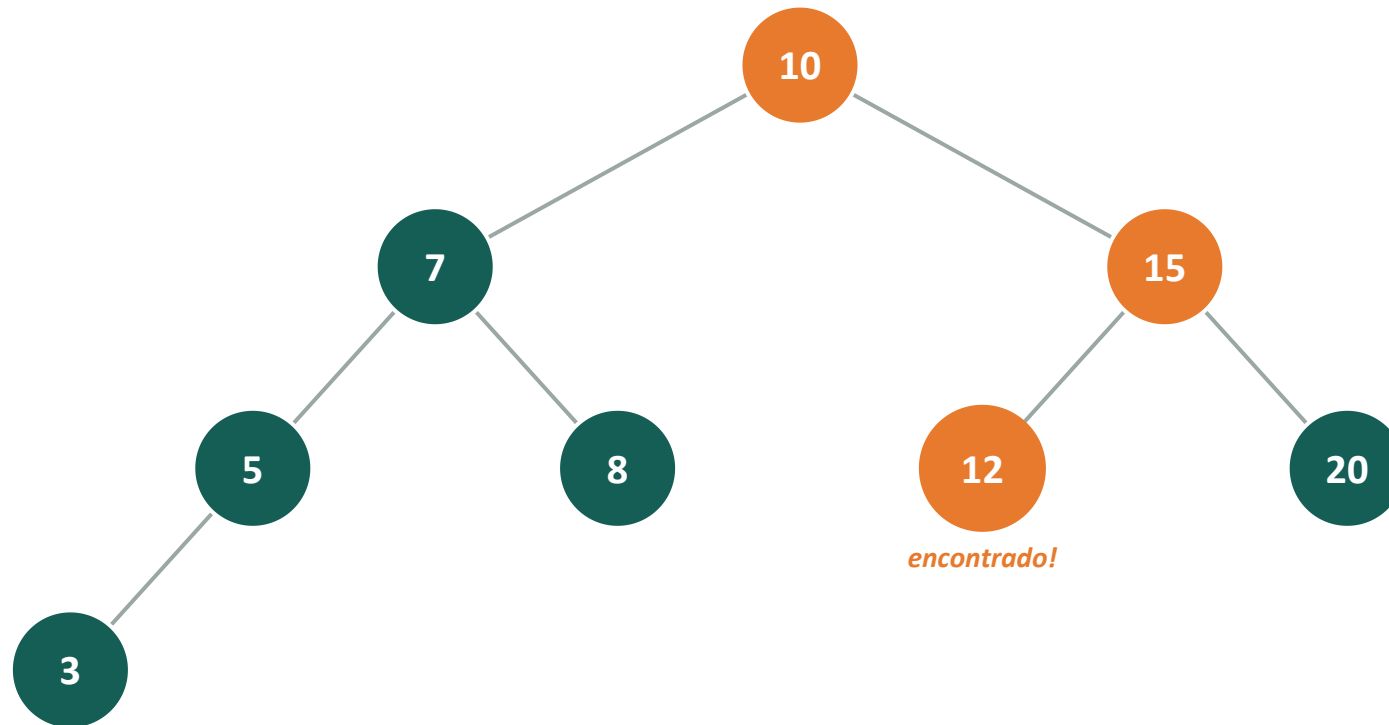
2

PARTE 2

Árvores de pesquisa binária

Menores à esquerda, maiores à direita.

Uma regra simples em cada nó



esquerda < nó < direita

em todos os nós da árvore

Pesquisar 12

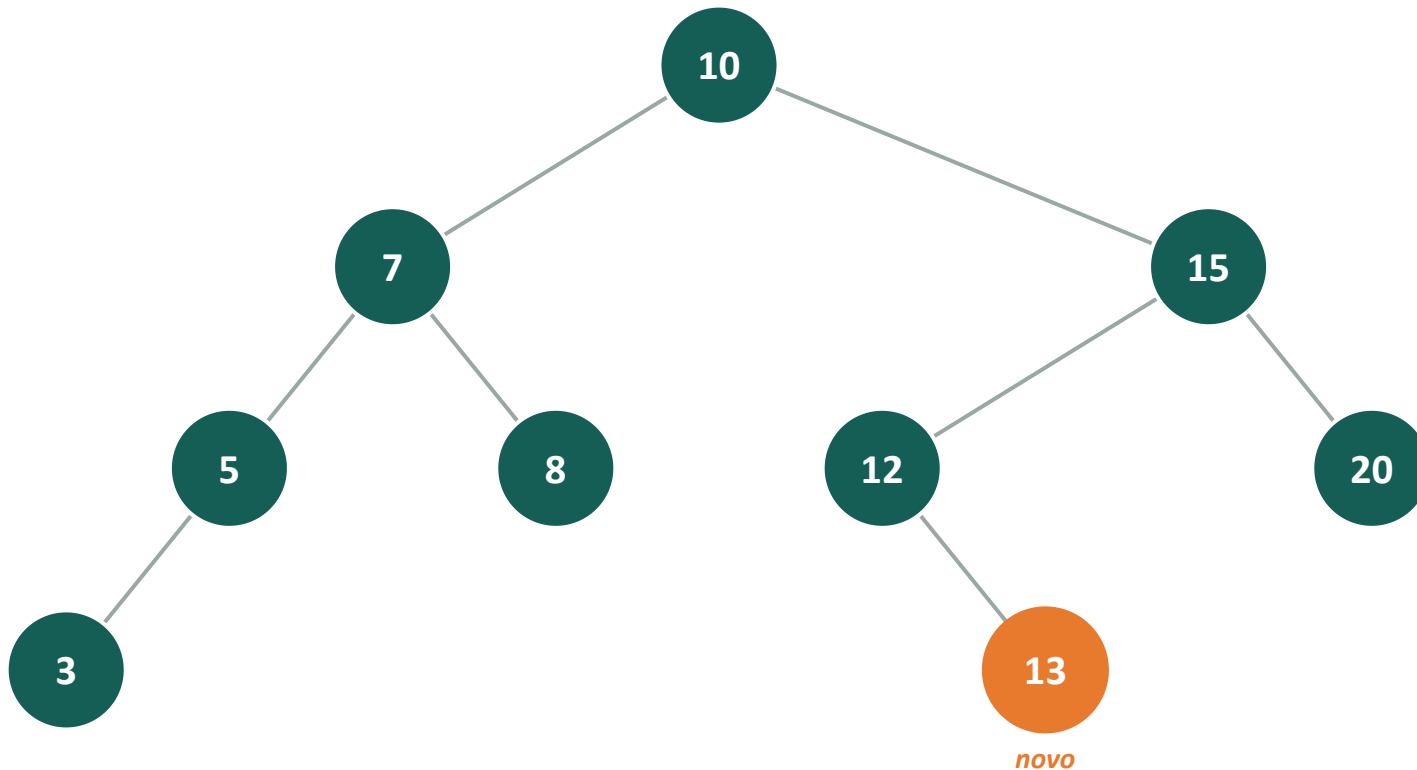
12 > 10 → direita

12 < 15 → esquerda

12 = 12 → encontrado

3 comparações para 8 elementos

Pesquisar até cair num lugar vazio



Inserir 13

$13 > 10 \rightarrow$ direita

$13 < 15 \rightarrow$ esquerda

$13 > 12 \rightarrow$ direita (vazio)

fica aqui, como folha

Bónus: percurso em ordem

esquerda $\rightarrow$ nó $\rightarrow$ direita

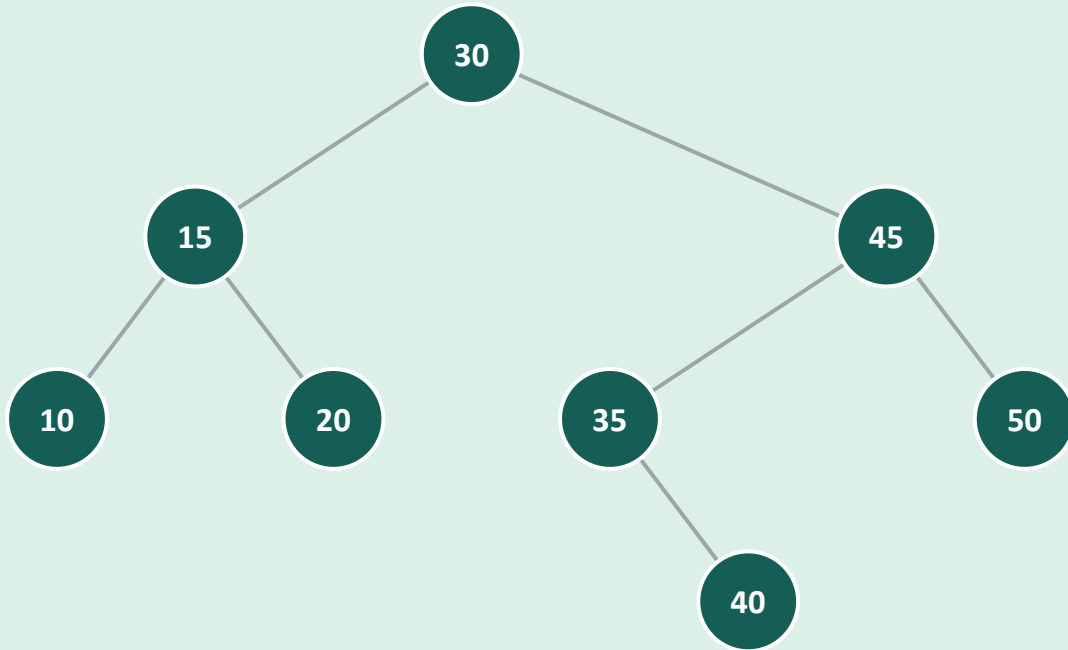
3 5 7 8 10 12 13 15 20

sai tudo ordenado!

A mesma regra, duas árvores muito diferentes

Equilibrada

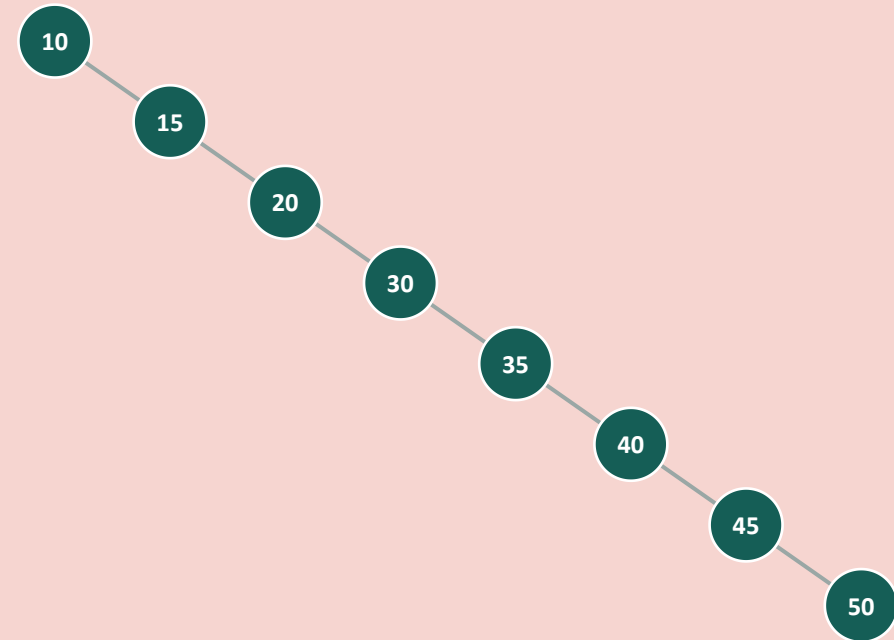
$O(\log n)$



altura 3 para 8 valores

Degenerada

$O(n)$



inserção por ordem: 10, 15, 20, 30... → é uma lista!

Uma BST com 1 000 000 de valores. Quantas comparações, no pior caso, para encontrar um valor?

Se estiver equilibrada

≈ 20

$\log_2 1\,000\,000 \approx 20$

Se estiver degenerada

1 000 000

percorre tudo, como uma lista

A diferença não está nos dados — está na forma da árvore.

Árvores que se equilibram sozinhas

Perfeitamente equilibrada

Subárvores sempre com a mesma profundidade.

Caro de manter em cada inserção e remoção.

Quase equilibrada

Diferenças pequenas e controladas entre caminhos.

Barato de manter e ainda garante $O(\log n)$.

Duas soluções clássicas de equilíbrio aproximado:

Árvores AVL

Controlam a diferença de alturas em cada nó.

Árvores Red-Black

Controlam o equilíbrio com cores. É a de hoje — e a do TreeMap do Java.

3

PARTE 3

Árvores Red-Black

Cinco regras e duas cores.

As regras do jogo

1 Cada nó é vermelho ou preto.

As duas cores.

2 A raiz é sempre preta.

Começa-se sempre pelo preto.

3 As folhas nulas (NIL) são pretas.

Os "lugares vazios" contam como pretos.

4 Um nó vermelho não tem filhos vermelhos.

Nunca dois vermelhos seguidos.

5 Todos os caminhos de um nó até às folhas têm o mesmo número de nós pretos.

A "altura preta" é igual em todo o lado.

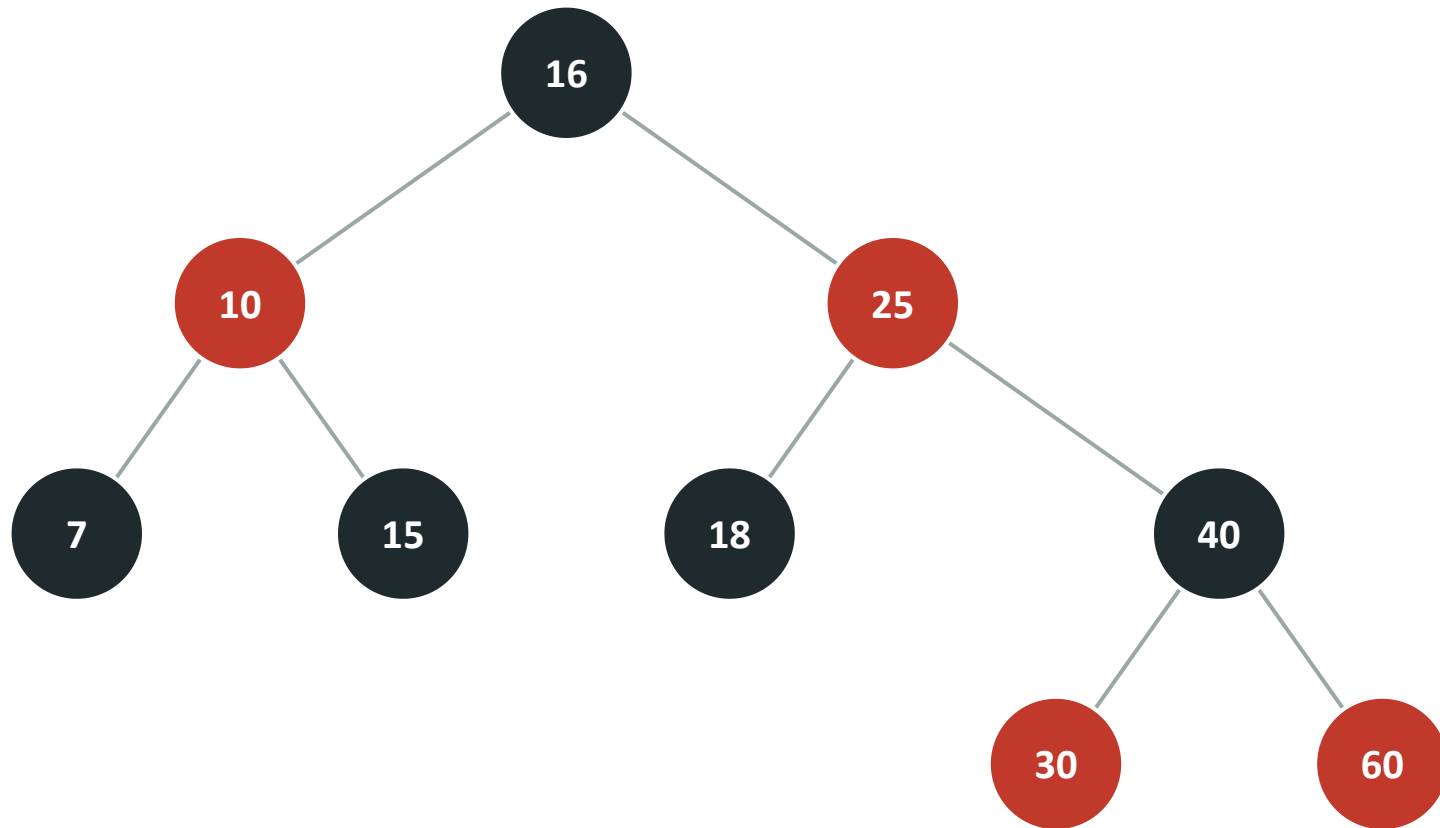
As duas que importam

Regra 4: não há dois vermelhos seguidos.

Regra 5: o número de pretos é igual em todos os caminhos.

Juntas, impedem a árvore de ficar torta.

Inserindo 10, 18, 7, 15, 16, 30, 25, 40, 60



Confirme as regras

Raiz preta?

16 ✓

Dois vermelhos seguidos?

nenhum ✓

Pretos por caminho

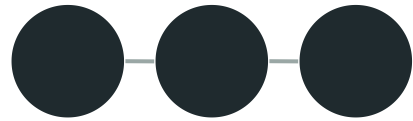
2 em todos ✓

Ex.: 16 → 25 → 40 → 30 passa por 16 e 40: dois pretos.

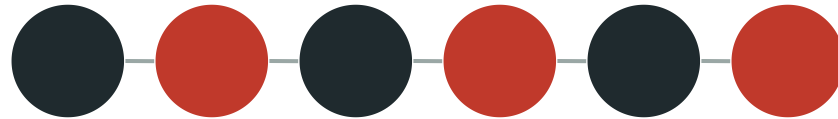
O caminho mais longo nunca passa do dobro do mais curto

Com 3 nós pretos em cada caminho:

mais curto: 3



mais longo: 6



$$\text{altura} \leq 2 \cdot \log_2(n + 1)$$

Logo, pesquisar, inserir e remover custam sempre $O(\log n)$ — mesmo no pior caso.

Com 1 000 000 de nós

$$\leq 40$$

níveis, no máximo

Uma árvore degenerada poderia ter 1 000 000.

4

PARTE 4

Inserção

Recolorir e rodar, caso a caso.

Inserir como numa BST — e pintar de vermelho



Inserir como numa BST

Desce-se pela árvore e o novo nó fica como folha, no lugar certo.



Pintar de vermelho

O novo nó é sempre vermelho (se for a raiz, fica preto).

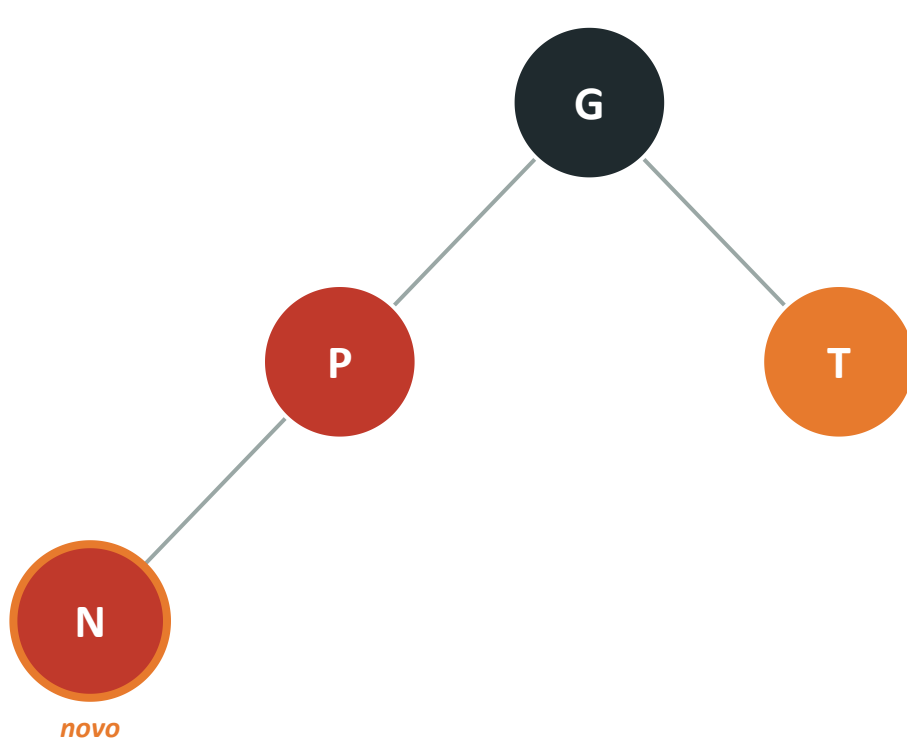


O pai é preto?

Então está tudo bem: não há nada a corrigir.

Porquê vermelho? Um vermelho não muda o número de pretos em nenhum caminho: a regra 5 fica garantida. O único problema possível é o pai também ser vermelho — dois vermelhos seguidos.

Quando o pai é vermelho, olha-se para o tio

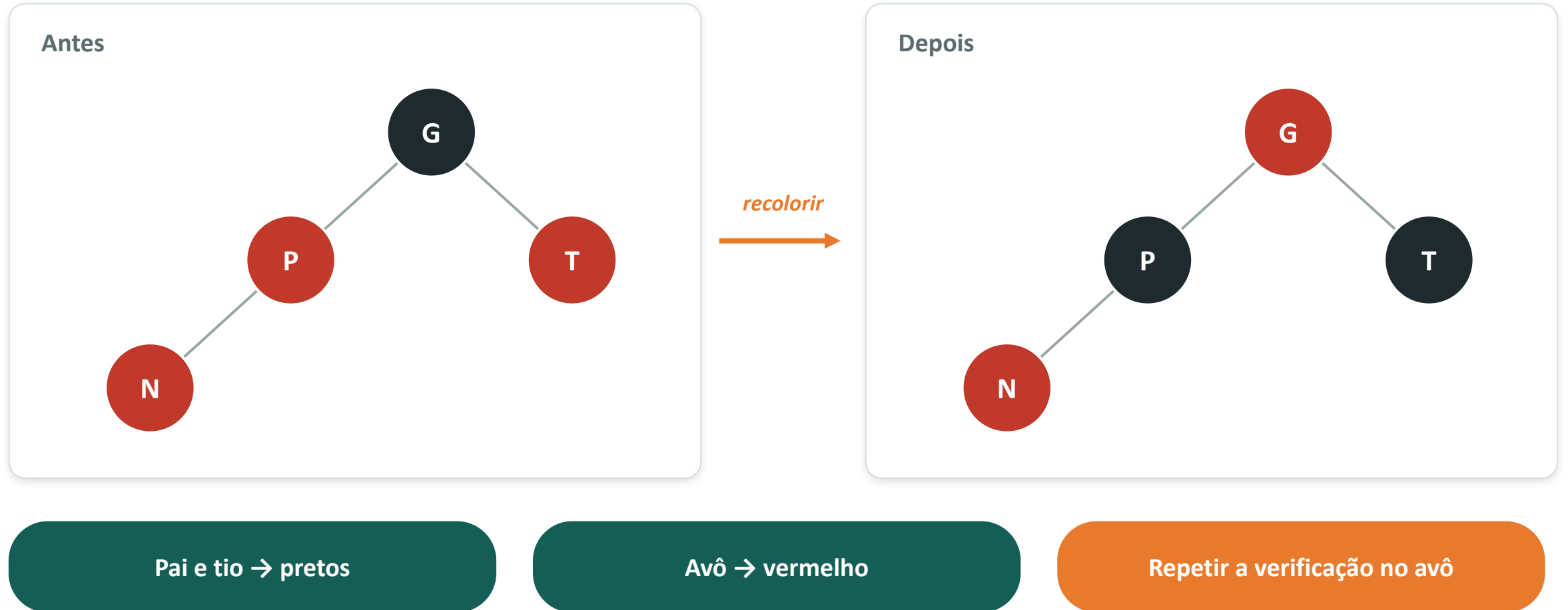


- N** o novo nó (vermelho)
- P** o pai (vermelho → conflito!)
- G** o avô (preto)
- T** o tio: irmão do pai

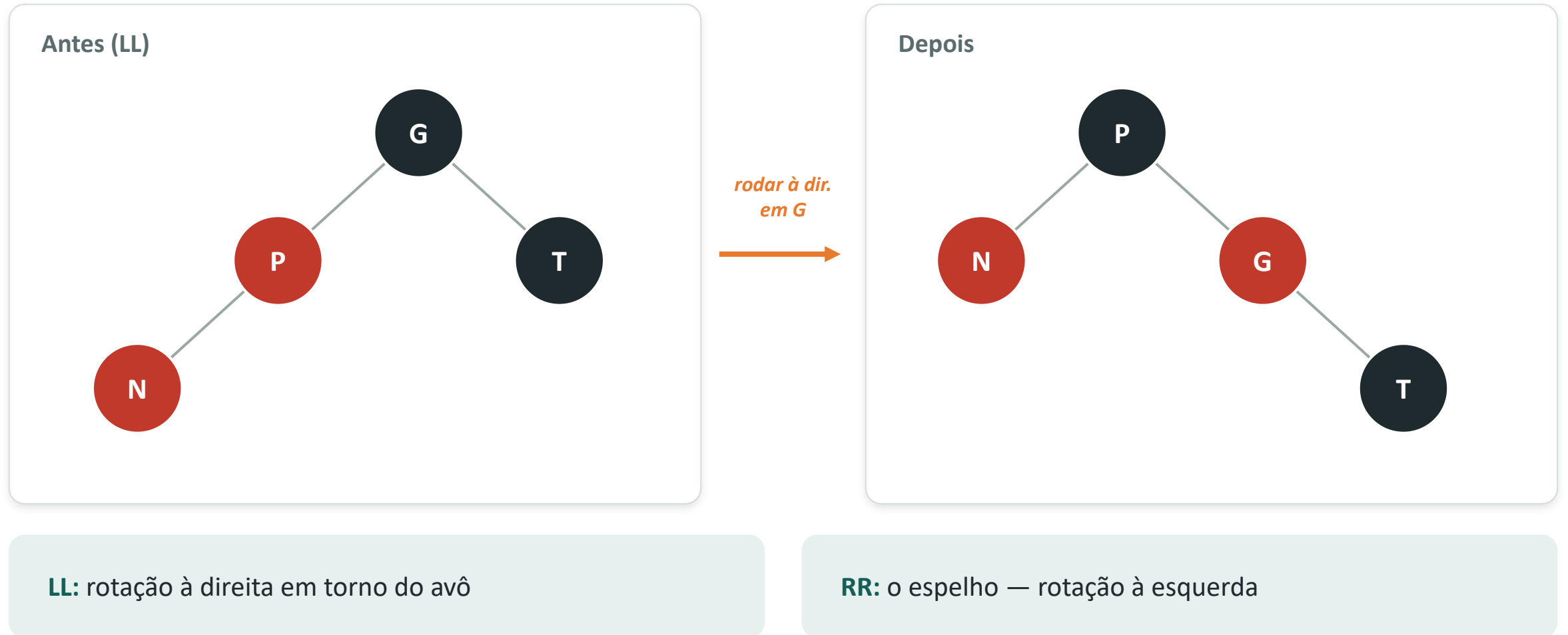
Tio vermelho
→ recolorir

Tio preto ou NIL
→ rodar

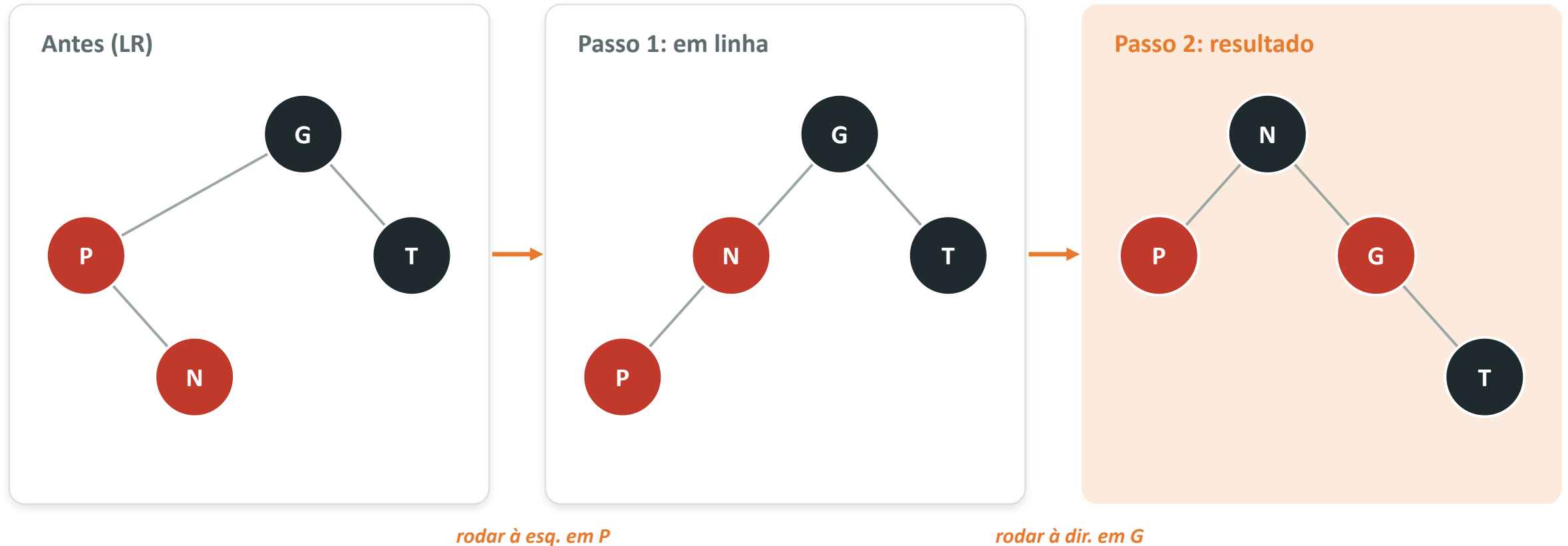
Recolorir e subir o problema



Uma rotação e troca de cores

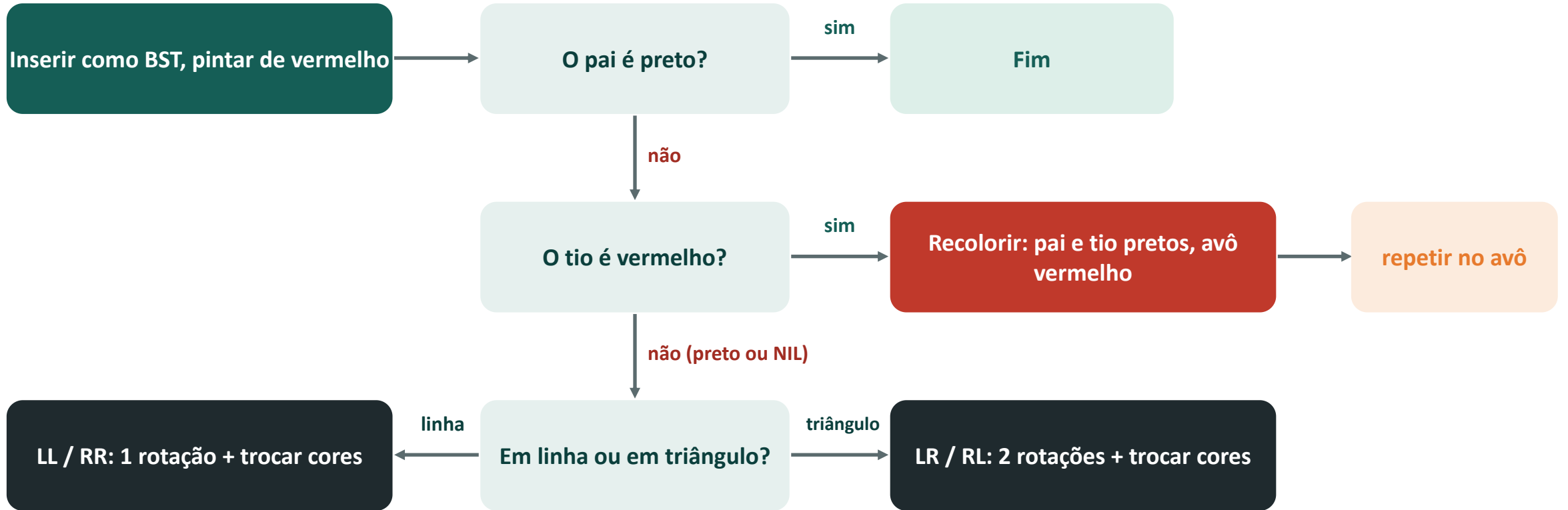


Duas rotações: primeiro endireitar, depois rodar



RL: o espelho — rotação à direita em torno do pai, depois à esquerda em torno do avô.

Toda a inserção num só diagrama



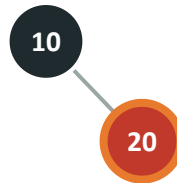
No fim: se a raiz ficou vermelha, pinta-se de preto.

Inserir 10, 20, 30, 15, 25, 27

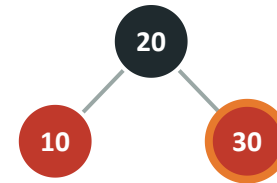
+10 raiz → preta



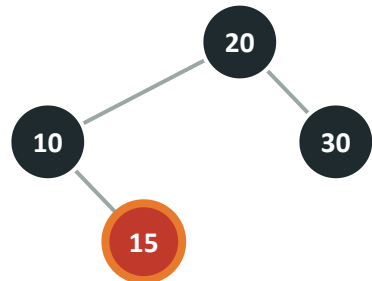
+20 pai preto: nada a fazer



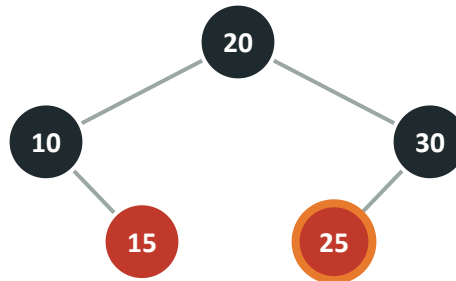
+30 RR: rotação à esquerda



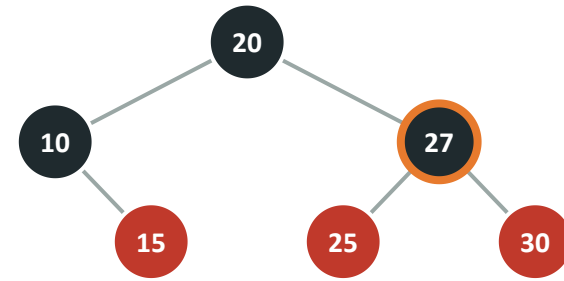
+15 tio vermelho: recolorir



+25 pai preto: nada a fazer



+27 LR: duas rotações



Um simulador para ver as rotações a acontecer

Red/Black Tree Visualization

www.cs.usfca.edu/~galles/visualization/RedBlack.html

1

Insira valores

Um a um, no campo "Insert".

2

Abrande a animação

Use o controlo de velocidade para ver cada passo.

3

Preveja primeiro

Antes de inserir, diga que caso vai acontecer. Depois confirme.

Em papel, passo a passo

1

Desenhe a árvore Red-Black que resulta de inserir, por esta ordem:

30, 86, 76, 8, 60, 13, 66, 61,
17

Apresente a árvore depois de cada inserção.

2

Desenhe a árvore Red-Black que resulta de inserir, por esta ordem:

96, 78, 6, 18, 7, 54, 25, 53,
36

Apresente a árvore depois de cada inserção.

Método: para cada valor, siga o mapa de decisão — pai preto? tio vermelho? em linha ou em triângulo? Confirme depois no simulador.

Agora em Java

3

Analisar e testar

Considere a implementação em Java de uma árvore Red-Black disponível no Moodle, que permite inserir e pesquisar inteiros. Analise o código e teste-o com os exemplos dos diapositivos.

Onde estão, no código, os três casos da inserção?

4

Mostrar a árvore

Acrescente um método que imprima a árvore. Para o exemplo dos valores 10, 18, 7, ...:

```
└─ 16 [B]
   └─ 10 [R]
      └─ 7 [B]
         └─ 15 [B]
      └─ 25 [R]
         └─ 18 [B]
            └─ 40 [B]
               └─ 30 [R]
                  └─ 60 [R]
```

Três ideias a reter

1

BST

Esquerda < nó < direita. $O(\log n)$ só se estiver equilibrada; senão, $O(n)$.

2

Red-Black

Raiz preta, sem vermelhos seguidos, mesmo nº de pretos em todos os caminhos → $O(\log n)$ garantido.

3

Inserir

Novo nó vermelho. Tio vermelho → recolorir. Tio preto → rodar (1 ou 2 vezes).

Próxima aula: remover nós de uma árvore Red-Black.

Bibliografia

1

Weiss, M. A. (2012). *Data Structures and Algorithm Analysis in Java* (3.^a ed.). Pearson.

2

Galles, D. *Red/Black Tree Visualization*. University of San Francisco. cs.usfca.edu/~galles/visualization/RedBlack.html

Dúvidas? Prof. Doutor Luís Simões da Cunha · luiscunha@ismt.pt